

How to interpret dplyr message `summarise()` regrouping output by 'x' (override with `.groups` argument)?

Asked 4 years, 2 months ago Modified 1 year, 1 month ago Viewed 185k times  Part of R Language Collective



I started getting a new message (see post title) when running `group_by` and `summarise()` after updating to dplyr development version 0.8.99.9003.

205



Here is an example to recreate the output:



```
library(tidyverse)
library(hablar)
df <- read_csv("year, week, rat_house_females, rat_house_males,
mouse_wild_females, mouse_wild_males
                2018,10,1,1,1,1,1
                2018,10,1,1,1,1,1
                2018,11,2,2,2,2,2
                2018,11,2,2,2,2,2
                2019,10,3,3,3,3,3
                2019,10,3,3,3,3,3
                2019,11,4,4,4,4,4
                2019,11,4,4,4,4,4") %>%
  convert(chr(year,week)) %>%
  mutate(total_rodents = rowSums(select_if(., is.numeric))) %>%
  convert(num(year,week)) %>%
  group_by(year,week) %>% summarise(average = mean(total_rodents))
```

The output tibble is correct, but this message appears:

```
summarise() regrouping output by 'year' (override with .groups argument)
```

How should this be interpreted? Why does it report regrouping only by 'year' when I grouped by both year and week? Also, what does it mean to override and why would I want to do that?

I don't think the message indicates a problem because it appears throughout the dplyr vignette: <https://cran.r-project.org/web/packages/dplyr/vignettes/programming.html>

I believe it is a new message because it has only appeared on very recent SO questions such as [How to melt pairwise.wilcox.test output using dplyr?](#) and [R Aggregate over multiple columns](#) (neither of which addresses the regrouping/override message).

Thank you!

   [Edit tags](#)

[Share](#) [Edit](#) [Follow](#) [Close](#) [Flag](#) [Protect](#)

edited Jun 16, 2020 at 1:34

asked Jun 1, 2020 at 20:26



Susie Derkins

2,599 2 14 21

7 Answers

Sorted by:
[Reset to default](#)

[Date modified \(newest first\)](#)



1



This is explained in <https://r4ds.hadley.nz/data-transform.html#grouping-by-multiple-variables>

When you summarize a tibble grouped by more than one variable, each summary peels off the last group. **In hindsight, this wasn't a great way to make this function work, but it's difficult to change without breaking existing code.** To make it obvious what's happening, dplyr displays a message

Share Edit Follow Flag

answered Jul 25, 2023 at 21:34



Kevin Murphy

31 1



268



It is just a friendly warning message about the resulting grouping structure; your output is correct. By default, if there is any grouping *before* the `summarise`, it drops one group variable i.e. the last one specified in the `group_by`. If there is only one grouping variable, there won't be any grouping attribute *after* the `summarise`. If there are more than one, the grouping is reduced by 1. So in your example since the input to `summarise` had two variables, the attribute for grouping is reduced to one, i.e. the resulting data frame would have 'year' as the grouping attribute.

As a reproducible example:

```
library(dplyr)
mtcars %>%
  group_by(am) %>%
  summarise(mpg = sum(mpg))
# `summarise()` ungrouping output (override with `.groups` argument)
# A tibble: 2 x 2
#   am   mpg
#<dbl> <dbl>
#1     0  326.
#2     1  317.
```

The message is that it is ungrouping i.e. when there is a single `group_by`, it drops that grouping after the `summarise`

```
mtcars %>%
  group_by(am, vs) %>%
  summarise(mpg = sum(mpg))
# `summarise()` regrouping output by 'am' (override with `.groups` argument)
# A tibble: 4 x 3
# Groups:   am [2]
#   am   vs   mpg
#<dbl> <dbl> <dbl>
#1     0     0  181.
#2     0     1  145.
#3     1     0  118.
#4     1     1  199.
```

Here, it drops the last grouping and regroup with the 'am'

If we check the `?summarise`, there is `.groups` argument which by default is "drop_last" and the other options are "drop", "keep", "rowwise"

`.groups` - Grouping structure of the result.

"drop_last": dropping the last level of grouping. This was the only supported option before version 1.0.0.

"drop": All levels of grouping are dropped.

"keep": Same grouping structure as .data.

"rowwise": Each row is its own group.

When .groups is not specified, you either get "drop_last" when all the results are size 1, or "keep" if the size varies. In addition, a message informs you of that choice, unless the option "dplyr.summarise.inform" is set to FALSE.

i.e. if we change the .groups in summarise, we don't get the message because the group attributes are removed

```
mtcars %>%
  group_by(am) %>%
  summarise(mpg = sum(mpg), .groups = 'drop')
# A tibble: 2 x 2
#   am   mpg
#* <dbl> <dbl>
#1     0  326.
#2     1  317.

mtcars %>%
  group_by(am, vs) %>%
  summarise(mpg = sum(mpg), .groups = 'drop')
# A tibble: 4 x 3
#   am   vs   mpg
#* <dbl> <dbl> <dbl>
#1     0     0  181.
#2     0     1  145.
#3     1     0  118.
#4     1     1  199.

mtcars %>%
  group_by(am, vs) %>%
  summarise(mpg = sum(mpg), .groups = 'drop') %>%
  str
#tibble [4 x 3] (S3: tbl_df/tbl/data.frame)
# $ am : num [1:4] 0 0 1 1
# $ vs : num [1:4] 0 1 0 1
# $ mpg: num [1:4] 181 145 118 199
```

Previously, this warning was not issued and it could lead to situations where the OP does a mutate or something else assuming there is no grouping and results in unexpected output. Now, the warning gives the user an indication that we should be careful that there is a grouping attribute

NOTE: The .groups right now is experimental in its lifecycle. So, the behaviour could be modified in the future releases

Depending upon whether we need any further transformation of the data based on the same grouping variable (or not needed), we could select the different options in .groups .

Share Edit Follow Flag

edited Apr 9, 2023 at 13:20

answered Jun 1, 2020 at 20:41



qix

7,776 1 60 66



akrun

884k 38 572 686

-
- 34 ▲ What would be useful as well is to explain why this grouping attribute matters at all, because it is not obvious.
– [jangorecki](#) Jun 1, 2020 at 21:10
-
- 12 ▲ Does this mean that if you use `.groups = 'drop'` you don't have to use `ungroup()` before running certain other functions such as `case_when` or `rowSums`? – [Susie Derkins](#) Jun 2, 2020 at 20:55
-
- 12 ▲ @SusieDerkins If you are using `summarise`, and with `groups = 'drop'`, then the group attributes are not there, so you don't need to `ungroup` (at least in the current scenario until this behavior is changed in tidyverse) – [akrun](#) Jun 2, 2020 at 20:57
-
- 76 ▲ Oh! to silence the message (keeping the old "drop_last" default), do `options(dplyr.summarise.inform=F)`
– [Mike Lawrence](#) Jun 7, 2020 at 13:41
-
- 17 ▲ @MikeLawrence thanks! That's all I needed. It's a bit offputting that previously working code is suddenly throwing warnings (there should be no such thing as a *friendly* warning). – [Fluffy](#) Jul 10, 2020 at 10:53
-

To solve this use `summarise(avg_mpg = mean(mpg), .groups = "drop")`, dplyr actually interprets the result table as grouped, that's why he shows you that warning.

1

Share Edit Follow Flag

answered Jun 2, 2022 at 12:43

[Aymen Azoui](#)

554 4 7

This can be as a result of `summarise_all()` vs `summarise(across(everything())...` when you have 2 or more grouping columns

0

```
> tibble(gr1=c(1,1,2), gr2=c(1,1,2), val=1:3) %>%
  group_by(gr1, gr2) %>%
  summarise(across(everything(), mean))
```

```
# `summarise()` has grouped output by 'gr1'.
# You can override using the #`.groups` argument.
```

```
# A tibble: 2 x 3
# Groups:   gr1 [2]
   gr1  gr2  val
<dbl> <dbl> <dbl>
1     1     1  1.5
2     2     2   3
```

```
> tibble(gr1=c(1,1,2), gr2=c(1,1,2), val=1:3) %>%
+   group_by(gr1, gr2) %>%
+   summarise_all(mean)
# No warnings here
```

```
# A tibble: 2 x 3
# Groups:   gr1 [2]
   gr1  gr2  val
<dbl> <dbl> <dbl>
1     1     1  1.5
2     2     2   3
```

So, the warning meaning: despite `everything()`, some of the columns will be skipped (grouping ones) in `summarise()`



Paraphrasing the accepted answer, it is just a friendly *confusing* warning.

17

`summarise()` has grouped output by 'xxx'



should be read: the *output* is OK and contains **all grouping columns** as attributes, only the *grouping keys* may be limited.



Example of grouping `mtcars` by `cyl`, `am` calculating `mean(mpg)`

```
mtcars %>% group_by(cyl, am) %>% summarise(avg_mpg = mean(mpg))
`summarise()` has grouped output by 'cyl'. You can override using the '.groups'
argument.
# A tibble: 6 x 3
# Groups:   cyl [3]
   cyl    am avg_mpg
<dbl> <dbl> <dbl>
1     4     0   22.9
2     4     1   28.1
3     6     0   19.1
4     6     1   20.6
5     8     0   15.0
6     8     1   15.4
```

The warning is saying that in the output only the first of the original grouping keys was preserved using the default `.groups = "drop_last"`. See the line `# Groups: cyl [3]`.

Nevertheless, the attributes are complete, both `cyl` and `am` are defined.

Here a quick overview of the available option showing the result with the function `group_keys()`

```
mtcars %>% group_by(cyl, am) %>% summarise(avg_mpg = mean(mpg)) %>% group_keys()
`summarise()` has grouped output by 'cyl'. You can override using the '.groups'
argument.
# A tibble: 3 x 1
   cyl
<dbl>
1     4
2     6
3     8

mtcars %>% group_by(cyl, am) %>% summarise(avg_mpg = mean(mpg), .groups = "keep")
%>% group_keys()
# A tibble: 6 x 2
   cyl    am
<dbl> <dbl>
1     4     0
2     4     1
3     6     0
4     6     1
5     8     0
6     8     1

mtcars %>% group_by(cyl, am) %>% summarise(avg_mpg = mean(mpg), .groups = "drop")
%>% group_keys()
# A tibble: 1 x 0
```

The only visible consequence is while using a *cascading* summarization - the example below produce only one summary row as the group key were dropped.

```
mtcars %>% group_by(cyl, am) %>% summarise(avg_mpg = mean(mpg), .groups = "drop")
%>% summarise(min_avg_mpg = min(avg_mpg))
# A tibble: 1 x 1
  min_avg_mpg
    <dbl>
1    15.0
```

But as the *grouping attributes* are all available, it should be not a problem to *reset* the *group keys* as required using `group_by(cyl, am)` before the subsequent *summarization*.

Share Edit Follow Flag

answered Sep 30, 2021 at 13:18



Marmite Bomber

20.8k 4 29 56



2

The answer is explained in ?summarise: "When .groups is not specified, it is chosen based on the number of rows of the results: If all the results have 1 row, you get "drop_last". If the number of rows varies, you get "keep".".



Basically, you get such message when there is more than one option to be used as .groups= argument. The message warns you that one option has been used in the calculation of the statistics following the condition above: "drop_last" or "keep" for results with 1 or more rows, respectively. Let's say that in your pipeline for some reason you applied two or more grouping criteria but you still need to summarise the data all across values regardless grouping, this can be done by setting .group = 'drop'. Unfortunately, this is only in theory, because, as you can see in @akrun's example, statistic values remain de same, no matter which option was set in .group = (I applied these different options to one of my datasets and obtained same results and same dataframe structure ('grouping structure is controlled by the .group= argument...'). However, by specifying the argument .group, no message is printed.

The bottom line is that when using summarise, if not grouping criteria is used, the output statistic is calculated across all rows and therefore 'results have 1 row'. When one or more grouping criteria are used, the output statistic is calculated within each group and therefore 'the number of rows varies' depending on the number of groups in data frame.

Share Edit Follow Flag

answered Feb 23, 2021 at 23:06



cmoreno

35 7



-9

This post is hidden. It was flagged as spam or offensive content and [deleted](#) 1 year ago by [Community Bot](#).



Locked. This answer is not currently accepting new interactions. [Learn more](#).



This answer was marked as spam or rude or abusive and is therefore not shown - you can see [the revision history](#) for details.

Share Edit Follow Flag

answered Dec 16, 2022 at 4:55

2  Copied from stackoverflow.com/a/66342391/4826457 with spam link – Suraj Rao Dec 16, 2022 at 5:16


Comments disabled on deleted / locked posts / reviews
